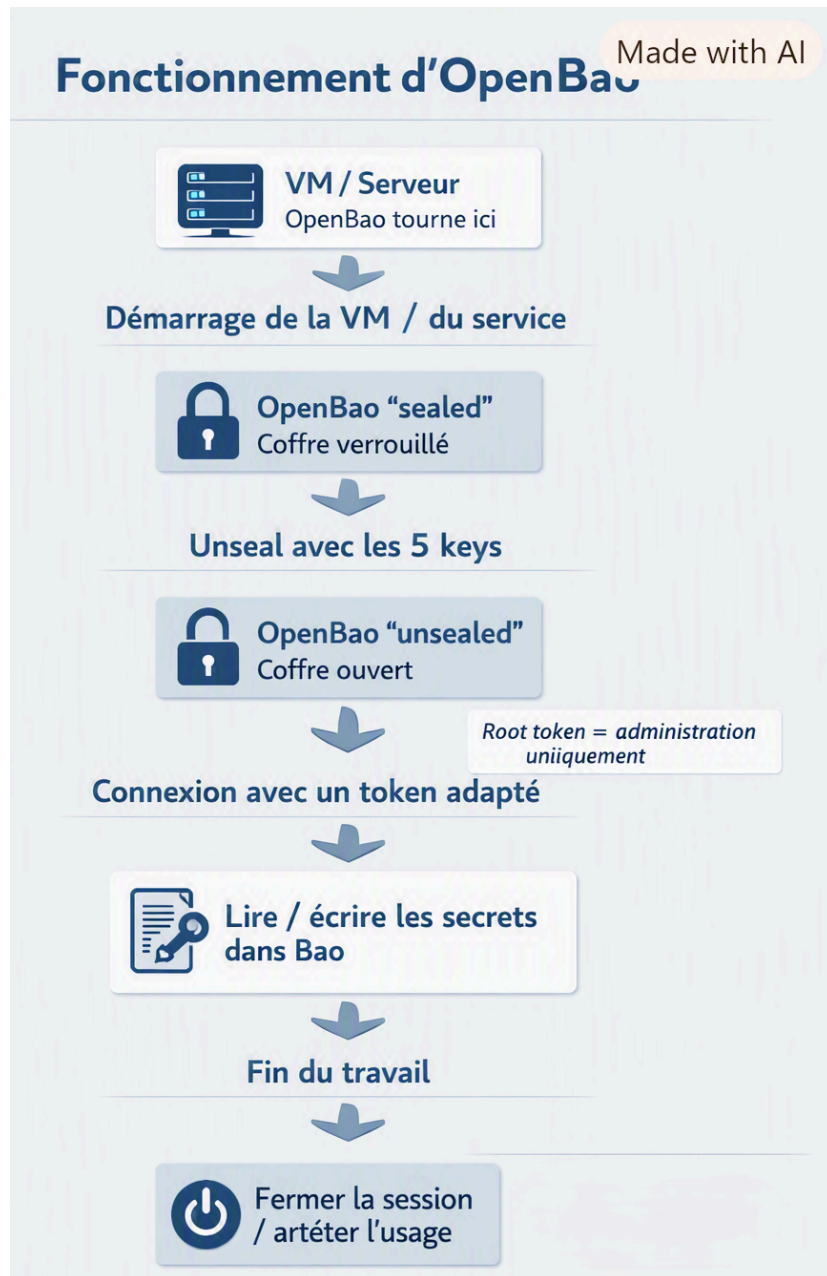


OpenBao (côté coffre et gestion des secrets)

Fiche 1 — OpenBao



Voici une **version finale nettoyée** de ta fiche 1, en gardant tout ce qui est important et en supprimant seulement les vrais doublons de rédaction et les parasites de mise en forme.

OpenBao

Objectif

Mettre en place une VM Debian servant de serveur OpenBao, puis y stocker les certificats, clés privées, CSR et chaînes PEM de plusieurs équipements réseau dans un coffre centralisé et chiffré.

L'objectif est aussi de pouvoir retrouver, relire et exporter ces informations si besoin, tout en gardant une copie de sauvegarde chiffrée hors de la VM.

1. Préparer la VM

Installer une Debian.

Après le premier démarrage, mettre le système à jour :

```
sudo apt update
sudo apt upgrade -y
```

Installer les outils utiles :

```
sudo apt install -y curl jq unzip
```

Créer un nom d'hôte simple, si besoin :

```
hostnamectl set-hostname OpenBao
```

2. Installer OpenBao

Vérifier si la commande `bao` existe déjà :

```
command -v bao
```

Récupérer la dernière version disponible via l'API GitHub, puis télécharger le bon fichier Linux amd64.

L'idée générale est :

- interroger l'API GitHub en ligne de commande,
- récupérer le nom du bon fichier Linux amd64,
- le télécharger avec `curl`,
- l'installer.

Exemple de récupération de version :

```
VERSION=$(curl -s https://api.github.com/repos/openbao/openbao/releases/latest | jq -r '.tag_name')
echo "$VERSION"
```

Récupérer l'URL du fichier Linux amd64 :

```
URL=$(curl -s https://api.github.com/repos/openbao/openbao/releases/latest | jq -r '.assets[] | select(.name | test("linux_amd64")) | .browser_download_url' | head -n 1)
echo "$URL"
```

Télécharger :

```
curl -LO "$URL"
```

Installer selon le format :

Si c'est un `.deb` :

```
sudo dpkg -i *.deb  
sudo apt -f install -y
```

Si c'est un `.tar.gz` :

```
tar -xzf *.tar.gz  
sudo install -m 0755 bao /usr/local/bin/bao
```

Vérifier :

```
bao version
```

3. Démarrer OpenBao

Lancer le service :

```
sudo systemctl start openbao  
sudo systemctl enable openbao  
sudo systemctl status openbao
```

Vérifier l'état du serveur :

```
export BAO_ADDR=https://127.0.0.1:8200  
export BAO_SKIP_VERIFY=true  
bao status
```

`BAO_ADDR` indique au client où parler à OpenBao.

`BAO_SKIP_VERIFY=true` désactive la vérification du certificat TLS côté client.

`bao status` vérifie que le coffre répond bien.

4. Initialiser OpenBao

Initialiser le coffre :

```
bao operator init
```

Cette commande crée :

- 5 clés d'unseal,

- 1 root token.

Il faut sauvegarder immédiatement ces informations dans un endroit sûr.

OpenBao reste verrouillé après cette étape.

5. Déverrouiller OpenBao

Déverrouiller le serveur avec 3 clés différentes :

```
bao operator unseal
```

Répéter la commande 3 fois avec 3 clés différentes.

Quand c'est bon, le statut doit indiquer que le coffre n'est plus scellé.

6. Activer le moteur KV

Après l'unseal, il faut s'authentifier pour pouvoir administrer OpenBao.

Dans notre cas, on utilise le root token, qui donne tous les droits nécessaires pour les premières opérations.

On peut définir ce token dans l'environnement avec :

```
export BAO_TOKEN="ton_root_token"
```

À partir de ce moment-là, toutes les commandes `bao` lancées dans le même terminal utilisent ce token.

C'est donc une forme d'authentification pratique et rapide, surtout pour l'administration locale.

Différence avec `bao login` :

- `export BAO_TOKEN=...` : on fournit directement le token au client.
- `bao login` : on passe par une commande de connexion plus interactive ou plus dépendante du mode d'authentification.

Dans notre cas, le résultat est le même : le client est authentifié avec un token valide.

Activer le moteur de secrets KV version 2 :

```
bao secrets enable --version=2 -path=secret kv
```

Cette commande :

- active le moteur `kv`,
- utilise la version 2,

- crée le point de montage nommé `secret`.

Ce moteur sert à stocker :

- clés privées,
- CSR,
- certificats,
- chaînes CA,
- infos d'inventaire.

Ce que signifie `secret`

`secret` n'est pas un dossier Linux classique.

C'est le nom du chemin sous lequel OpenBao expose ce moteur de secrets.

On peut créer plusieurs moteurs si besoin, avec des noms différents, par exemple :

- `secret/`
- `apps/`
- `lab/`

Chaque moteur a son propre espace de stockage logique.

Ce qu'il faut retenir

- `export BAO_TOKEN=...` permet au client `bao` d'utiliser le root token.
- Ce n'est pas une commande `bao login`, mais l'effet est le même pour l'authentification.
- Le root token sert à faire les premières opérations d'administration.
- `bao secrets enable --version=2 -path=secret kv` crée le moteur KV nommé `secret`.
- On peut créer plusieurs moteurs KV avec des noms différents si nécessaire.

7. Créer les dossiers sur la VM

Créer le dossier de travail :

```
mkdir -p ~/Certs
```

Créer aussi une arborescence si besoin :

```
mkdir -p ~/Certs/pki/switch/Netgear
```

Vérifier :

```
ls
```

ou :

```
dir
```

8. Copier les fichiers depuis le PC hôte

Depuis le PC hôte en PowerShell, se placer dans le dossier qui contient les fichiers puis les envoyer vers la VM avec `scp`.

Exemple :

```
scp Switch-Info2.cer celduc@192.168.1.44:~/Certs/
```

Exemple avec plusieurs fichiers :

```
scp Switch-Info2.cer Switch-Info2.csr switch_info2_distribution.key Switch-Info2.pem celduc@192.168.1.44:~/Certs/
```

Le dossier de réception sur la VM est donc :

```
~/Certs
```

9. Vérifier les fichiers sur la VM

Sur la VM Linux :

```
cd ~/Certs
ls -l
```

Compter les fichiers présents dans le dossier courant :

```
find . -maxdepth 1 -type f | wc -l
```

10. Stocker les fichiers dans OpenBao

Une fois les fichiers présents sur la VM, les enregistrer dans OpenBao avec `bao kv put`.

Le `@` sert à dire : prends la valeur dans le fichier. Donc dans ton exemple, OpenBao stocke le contenu des fichiers `switch_info2_distribution.key`, `Switch-Info2.csr`, etc., pas leurs noms.

Exemple pour l'entrée `Switch-Methodes` :

```
bao kv put secret/pki/switch/Netgear/Switch-Methodes \
  key=@switch_info2_distribution.key \
  csr=@Switch-Info2.csr \
  cert=@Switch-Info2.cer \
  chain=@Switch-Info2.pem \
  info="Switch-Methodes.celduc.lan"
```

Exemple pour l'entrée `Switch-BE` :

```
bao kv put secret/pki/switch/Netgear/Switch-BE \
  key=@Switch-BE.key \
  csr=@Switch-BE.csr \
  cert=@Switch-BE.cer \
  chain=@Switch-BE.pem \
  info="Switch-BE.celduc.lan"
```

Exemple pour l'entrée `switch_info2_distribution` :

```
bao kv put secret/pki/switch/Netgear/switch_info2_distribution \
  key=@switch_info2_distribution.key \
  csr=@switch_info2_distribution.csr \
  cert=@switch_info2_distribution.celduc.lan.cer \
  info="switch_info2_distribution.celduc.lan"
```

11. Ce que signifie la sortie

Quand `bao kv put` réussit, tu obtiens quelque chose comme :

```
Secret Path
secret/data/pki/switch/Netgear/Switch-BE
```

et une metadata avec :

- `created_time`,
- `deletion_time`,
- `destroyed`,
- `version`.

Cela signifie que :

- l'entrée a bien été créée,
- la version 1 ou 2 a bien été stockée,
- si une nouvelle écriture est faite au même chemin, une nouvelle version sera créée.

12. Vérifier une entrée

Pour voir ce qui a déjà été créé :

```
bao kv list secret/pki/switch/Netgear/
```

Exemple de résultat :

```
Switch-BE
Switch-Info2
```

Switch-Methodes

switch_info2_distribution

Pour voir une entrée précise :

```
bao kv get secret/pki/switch/Netgear/Switch-Methodes
```

13. Récupérer un champ précis

Pour lire uniquement un champ, par exemple la clé privée :

```
bao kv get -field=key secret/pki/switch/Netgear/Switch-Methodes
```

Autres champs possibles :

```
bao kv get -field=csr secret/pki/switch/Netgear/Switch-Methodes
bao kv get -field=cert secret/pki/switch/Netgear/Switch-Methodes
bao kv get -field=chain secret/pki/switch/Netgear/Switch-Methodes
bao kv get -field=info secret/pki/switch/Netgear/Switch-Methodes
```

Tu peux aussi rediriger le contenu dans un fichier local :

```
bao kv get -field=key secret/pki/switch/Netgear/Switch-Methodes > key.pem
```

14. Supprimer une entrée

Pour supprimer une entrée précise :

```
bao kv delete secret/pki/switch/Netgear/Switch-Methodes
```

Cela supprime seulement cette entrée, pas tout OpenBao.

15. Comptage des fichiers

Sous Linux :

```
find . -maxdepth 1 -type f | wc -l
```

Sous PowerShell :

```
(Get-ChildItem -File).Count
```

16. Sauvegarde complémentaire

En plus d'OpenBao, tu peux garder une copie de sauvegarde sur une clé USB, mais chiffrée.

Le plus propre est :

- copier le dossier source sur la clé USB,

- chiffrer la clé ou un conteneur avec VeraCrypt,
- garder le mot de passe en lieu sûr,
- démonter le volume après usage.

Le principe est :

- OpenBao = stockage principal,
- clé USB chiffrée = sauvegarde de secours,
- suppression des copies locales sur le PC hôte seulement après validation.

17. Ce qu'il faut retenir

- `mkdir -p` crée les dossiers s'ils n'existent pas.
- `scp` copie les fichiers du PC hôte vers la VM.
- `bao kv put` enregistre les fichiers dans OpenBao.
- `bao kv get` relit une entrée.
- `bao kv get -field=...` extrait un champ précis.
- `bao kv list` affiche les entrées existantes.
- `bao kv delete` supprime une entrée.
- OpenBao garde des versions si tu réécrit au même chemin.
- Une entrée différente = un chemin différent.
- `Switch-BE` ne remplace pas `Switch-Methodes` si le chemin est différent.

18. Phrase de conclusion

Une fois les fichiers transférés sur la VM Linux, ils sont stockés dans OpenBao via `bao kv put` sous des chemins distincts par équipement. Les entrées peuvent ensuite être listées, lues, exportées ou supprimées selon les besoins, tandis qu'une sauvegarde complémentaire peut être conservée sur une clé USB chiffrée avec VeraCrypt.

Si tu veux, je peux aussi te faire juste après la **même version finale pour la fiche 2**, au même niveau de propreté.